Yes — this is very doable as a mini project, but I'd separate **"AI-assisted inventory generation"** from **"legally useful deposit-dispute evidence."** The AI can save you time, but the value in a TDS-style dispute is the reviewed, dated, comparable check-in/check-out evidence, not the fact that an AI detected a sofa.

Also, small arithmetic flag: if the agent is charging **£140 + VAT**, and standard UK VAT is **20%**, the VAT-inclusive total would normally be **£168**, not £165. Worth querying the quote. (GOV.UK)

Bottom line

I would **not** pay £140+VAT just because the agent says AI-free DIY is invalid. In Northern Ireland, the key point is that if there is a deposit dispute, the landlord has the burden of proving entitlement to deductions: Housing Rights NI says the deposit is treated as the tenant's money and the onus is on the landlord to prove a right to keep some of it. (Housing Rights) TDS NI also lists a check-in report/inventory and check-out report as evidence for showing the condition at the start and end of the tenancy. (tdsnorthernireland.com)

So your DIY system can be good enough **if** it produces a clear, dated, room-by-room inventory with photos/video, condition notes, tenant acknowledgement, and a matching check-out report later. mydeposits' 2026 guidance says useful dispute evidence includes signed start-of-tenancy inventories, check-in/check-out reports, date-stamped photos/videos, receipts/quotes, communications, and the tenancy agreement. (mydeposits)

Are there cheap tools already?

Yes, in three buckets.

Type	Examples	Fit for your idea
Property inventory/reporting apps	The Inventory Manager, Inventory Hive, InventoryBase, Reports2Go, Inventory Egg, HelloReport	Best for producing a professional-looking report with photos, signatures, check-in/check-out structure. Some are cheap; The Inventory Manager advertises inventory reports at **£5.95**. (The Inventory Manager)
AI property-inspection tools	InventoryBase BaseAI, KapturAI, Inspecto, VeriStay	Closest to your "video → inventory report" idea, but many are agent/professional-focused or require demo/pricing contact. InventoryBase says its AI turns inspection photos/notes into reports; KapturAI and Inspecto market video-to-report workflows. (InventoryBase)

Type	Examples	Fit for your idea
General home-inventory AI apps	Scanlily, HomeZada	Better for “walk around with a video and list objects” than for tenancy-dispute-grade condition reporting. Scanlily says it can use video recognition to identify/catalog items, and HomeZada says its video recognition detects furniture, electronics, appliances, fixtures and building materials. (Scanlily)

For a landlord inventory, I’d probably use a cheap property-reporting app **or** build your own report generator, then use AI only to pre-fill the boring parts.

Best technical approach for your mini app

SAM is impressive, but there’s a catch: **SAM/SAM 2 segments objects; it does not automatically know what every object is called.** Meta describes SAM 2 as a promptable segmentation model for images and videos: you click, box, or mask an object, then it can segment and track it through video. (Meta AI) For your goal, you need:

Object naming: “This is a sofa / radiator / hob / extractor fan.”
Object localisation: “Here it is in the frame.”
Tracking/deduplication: “This is the same sofa seen again 8 seconds later.”
Condition evidence: “Good condition; minor scuff on left arm; stain 4 cm on carpet.”
Report generation: “Room-by-room signed inventory with photos.”

So the model stack should be:

Video → frames → open-vocabulary detector/VLM → tracker/SAM → dedupe → human review → PDF report.

Good model choices:

Task	Cheap/local option	Cloud/easy option
Detect/nickname objects	YOLOE, Florence-2, Grounding DINO	Google Video Intelligence, AWS Rekognition, OpenAI/Gemini-style vision models
Segment/track objects	SAM 2 / Grounded SAM 2	Google Video Intelligence object tracking
Summarise conditions	Vision-language model + human edit	OpenAI/other VLM over selected frames
Report generation	Python + HTML/PDF	Inventory app export

YOLOE is a particularly interesting local route because it is an open-vocabulary detection/segmentation model: unlike older YOLO models limited to fixed categories, Ultralytics says YOLOE can use text, image, or internal-vocabulary prompts for real-time detection. (Ultralytics Docs) Grounded SAM 2 is also relevant because it combines Grounding DINO/Florence-style grounding with SAM 2 to "ground and track" objects in videos. (GitHub) Florence-2 is another lightweight vision-language model that can do captioning, object detection, grounding, and segmentation under a prompt-based interface. (OpenVINO Documentation)

Cheapest MVP route

For a first working version, I'd avoid training anything.

MVP 1: "Video to reviewed inventory list"

Input: One walkthrough video per property, ideally split by room.
Output: CSV/JSON + editable review page + PDF inventory.

Pipeline:

1. **Capture video**
   Record slowly, one room at a time. Start each room by saying: "Kitchen, 8 June 2026, check-in inventory." Pan walls, floor, ceiling, doors, windows, fixtures, then close-ups of appliances, damage, meters, keys, smoke alarms, and any supplied furnishings.
2. **Extract frames**
   Use FFmpeg to sample one frame every 1–2 seconds, plus a few high-quality stills.

```
ffmpeg -i walkthrough.mp4 -vf fps=1 frames/%06d.jpg
```

3. **Detect objects**
   For quickest results, try **Google Cloud Video Intelligence object tracking** first. Google says the object-tracking feature returns labels/tags, object locations in frames, and can track multiple detected objects. It also notes that very small objects may not be detected. (Google Cloud Documentation) Pricing is attractive for experiments: Google lists the first 1,000 minutes as free and object tracking at **$0.15/minute** after that. (Google Cloud)
4. **Generate a draft room inventory**
   Aggregate detections by room and object class. Example:

```
{
  "room": "Kitchen",
  "items": [
    {
      "name": "oven",
      "count": 1,
      "evidence_frames": ["00:01:12", "00:01:16"],
```

```
      "condition": "AI suggested: appears clean; human review required"
    },
    {
      "name": "extractor fan",
      "count": 1,
      "evidence_frames": ["00:01:20"],
      "condition": "Human to complete"
    }
  ]
}
```

5. **Human review UI**
   Build a simple Streamlit app:
   - left side: video frame / thumbnail
   - right side: room, item name, condition, cleanliness, defects
   - buttons: accept, rename, merge duplicate, delete false positive, add missing item
6. **Export report**
   Generate a PDF with:
   - property address
   - date/time
   - landlord/agent/tenant names
   - room-by-room contents
   - condition notes
   - embedded thumbnails
   - full video filename and SHA-256 hash
   - meter readings
   - keys issued
   - smoke/CO alarm checks
   - tenant signature or email acknowledgement

This MVP gives you a useful prototype without fighting every hard computer-vision problem at once.

More “AI nerd project” route

Once MVP 1 works, replace the cloud detector with a local stack:

MVP 2: Local open-source model stack

Option A: YOLOE-first

Use YOLOE with a prompt vocabulary such as:

```
sofa, armchair, dining table, chair, bed, mattress, wardrobe, drawer,
curtains,
blind, carpet, radiator, boiler, oven, hob, extractor fan, fridge, freezer,
washing machine, dishwasher, sink, tap, toilet, bath, shower, mirror, lamp,
smoke alarm, carbon monoxide alarm, door, window, key, meter
```

Then run tracking across frames and deduplicate.

Option B: Florence-2 / Grounding DINO + SAM 2

Use Florence-2 or Grounding DINO to find candidate objects, then SAM 2 for masks/tracking. This is closer to the fancy demos you've seen. Grounded SAM 2 already has video-object-tracking demos with custom video input and text prompts. (GitHub)

Option C: VLM-only for draft notes

For each room, send 5–10 carefully selected frames to a vision-language model and ask:

> "Create a landlord inventory draft. List visible fixtures, fittings, furniture, appliances, cleanliness, and visible defects. Do not invent anything. Mark uncertain items as uncertain."

This is much easier to build, but less structured. It is good for drafting prose, not for reliable object counts.

What the app should actually detect

For tenancy purposes, "objects" are not enough. A proper inventory needs the **condition of the property itself**, including fixed features. Total Landlord's guide describes a robust check-in inventory as including contents, each room's condition, fixtures, standard of cleanliness, and details from ceiling paint to bathroom fittings. (Total Landlord Insurance)

Your schema should include:

```
Property
  Rooms
    Surfaces: ceiling, walls, skirting, doors, windows, floors
    Fixtures: radiators, sockets, light fittings, smoke alarms, taps,
sanitaryware
    Appliances: oven, hob, extractor, boiler, fridge, washer
    Furniture: sofa, bed, mattress, table, chairs, wardrobes
    Cleanliness: clean / dusty / stained / greasy / mould present
    Defects: scratch, chip, dent, stain, crack, burn, missing item
    Evidence: photo/video frame IDs, timestamps, notes
```

Condition detection: keep it human-reviewed

This is the biggest limitation. AI can often see “sofa”, “radiator”, “stain”, or “crack”, but it will miss small scuffs, overstate harmless shadows, confuse reflections, and fail on white walls, carpets, and glossy appliances. Google’s own object-tracking docs note that very small objects may not be detected. (Google Cloud Documentation)

So I’d design condition as:

- AI suggests condition text.
- Human must confirm.
- Defects require close-up photo.
- Scratches/stains should include a ruler/coin/tape measure in the shot.
- The final report should say “reviewed and confirmed by landlord/agent,” not “AI-certified.”

Evidence pack design

To make this useful in a future dispute, include:

1. **Check-in report before tenancy starts**
   Date, address, room-by-room condition, photos, video reference, meter readings, keys, alarms.
2. **Tenant acknowledgement**
   Get the tenant to sign, or email them the report and ask them to reply confirming receipt and any corrections. mydeposits says an inventory carries maximum weight when signed by both landlord and tenant, and at minimum there should be evidence that the tenant was sent it and had time to comment. (mydeposits)
3. **Immutable-ish evidence trail**
   Store original videos, extracted frames, and a hash file:

   ```
   shasum -a 256 walkthrough.mp4 > evidence_hashes.txt
   shasum -a 256 report.pdf >> evidence_hashes.txt
   ```
4. **Matching check-out process**
   At the end, record the same rooms in the same order. Total Landlord recommends using a consistent format for check-in and check-out, and says both reports are needed as evidence if costs/deductions are disputed. (Total Landlord Insurance)

Suggested build plan

Phase 0 — Don’t build yet: create your capture template

Before coding, make a simple checklist. This matters more than the model.

Rooms:

```
Entrance / hallway
Living room
Kitchen
Bedroom 1
Bedroom 2
Bathroom
Exterior / garden / bins
Meters / boiler / fuse box
Keys / alarms / certificates
```

Per room:

```
Ceiling
Walls
Floor
Doors
Windows
Lighting
Heating
Furniture
Appliances
Cleanliness
Existing defects
Photos required
```

Phase 1 — Working prototype

Goal: upload video → get draft object list.

Stack:

```
Python
FFmpeg
OpenCV
Google Video Intelligence API or AWS Rekognition
SQLite
Streamlit
Pandas
Jinja2 / WeasyPrint / ReportLab for PDF
```

Use Google first because its video API already supports object tracking with labels, locations and timestamps. (Google Cloud Documentation) AWS Rekognition is also viable: its pricing

example for image label detection shows **$0.001 per image** for the first million images, and its free tier includes 60 free minutes of video analysis per month for 12 months. (Amazon Web Services, Inc.)

Phase 2 — Review UI

Goal: make it usable.

Features:

```
Room selector
Frame thumbnails
Detected item table
Rename / merge / delete detections
Add missing item
Condition dropdown
Defect notes
Photo evidence selector
Export PDF
```

Condition dropdown:

```
New
Good
Fair
Poor
Marked
Scratched
Stained
Cracked
Chipped
Dirty
Not tested
Not visible
```

Phase 3 — Local model replacement

Goal: reduce cloud dependency and learn the fun CV part.

Try:

```
YOLOE for open-vocabulary object detection
Florence-2 for object/caption prompts
Grounding DINO for text-prompted detection
```

```
SAM 2 for segmentation/tracking
CLIP/image embeddings for deduplication
```

This is where your SAM idea comes in. Use SAM 2 to track selected objects through the video, but use YOLOE/Grounding DINO/Florence-2 to name them.

Phase 4 — Check-in/check-out comparison

Goal: landlord-useful product.

At check-out, the app should show:

```
Kitchen > Oven
Check-in: clean, good condition, photo A
Check-out: grease marks, handle loose, photo B
Suggested issue: cleaning / repair
Landlord decision: claim / no claim / fair wear and tear
Evidence: before/after image pair
```

This is much more useful than a raw object list.

Rough cost comparison

For one normal property, say 15–25 minutes of walkthrough video:

Option	Likely cost	Notes
Agent inventory	~£168 if £140 + 20% VAT	Query their £165 total.
Google Video Intelligence MVP	Often £0 during free allowance; after free tier, object tracking listed at $0.15/min	Quickest API prototype. (Google Cloud)
AWS Rekognition frame sampling	Roughly cents to a few dollars if you sample frames, depending on frame count/free tier	Simple labels, less tenancy-specific. (Amazon Web Services, Inc.)
The Inventory Manager	£5.95 per inventory report listed	Likely manual/reporting workflow rather than full AI video detection. (The Inventory Manager)
Local open-source	£0 software, but time/GPU cost	Best learning project; more setup.

My recommended approach

Build the app, but use it to create a **reviewed evidence report**, not an “AI-certified inventory.”

Best practical route:

1. Use a cheap inventory/reporting tool or your own PDF template for the final document.
2. Use Google Video Intelligence or AWS Rekognition for the first prototype.
3. Use YOLOE + SAM 2 / Grounded SAM 2 later for the fun open-source version.
4. Keep human review mandatory.
5. Get tenant acknowledgement by signature or email.
6. Repeat the same process at check-out.

That gives you the best of both worlds: you avoid paying the agent for a commodity report, you learn a genuinely interesting AI/video pipeline, and you still produce the sort of evidence that matters if there is ever a deposit dispute.